

Op Overload

重载运算符

重载运算符是通过重新定义运算符，使得其支持特定数据类型的运算操作。重载运算符是重载函数的特殊情况。

当一个运算符出现在一个表达式中，并且运算符的至少一个操作数具有一个类或枚举的类型时，则使用重载决议（overload resolution）确定应该调用哪个满足相应声明的用户定义函数。¹

通俗的讲，如果把使用「运算符」看作一个调用特殊的函数（如将 `1+2` 视作调用 `add(1, 2)`），并且这个函数的参数（操作数）至少有一个是 `class`、`struct` 或 `enum` 的类型，编译器就需要根据操作数的类型决定应当调用哪个自定义函数。

在 C++ 中，我们可以重载几乎所有可用的运算符。

▼ 一些可重载运算符的列举

一元运算：+（正号）；-（负号）；~（按位取反）；++；--；！（逻辑非）；*（取指针对应值）；&（取地址）；->（类成员访问运算符）等。

二元运算：+；-；&（按位与）；[]（取下标）；==；=（赋值）等。

其它：()（函数调用）；""（后缀标识符¹，C++11 起）；new（内存分配）；,（逗号运算符）；<=>（三路比较²，C++20 起）等。

限制

重载运算符存在如下限制：

- 只能对现有的运算符进行重载，不能自行定义新的运算符。
- 以下运算符不能被重载：`::`（作用域解析），`.`（成员访问），`.*`（通过成员指针的成员访问），`?:`（三目运算符）。
- 重载后的运算符，其运算优先级，运算操作数，结合方向不得改变。
- 对 `&&`（逻辑与）和 `||`（逻辑或）的重载失去短路求值。

实现

重载运算符分为两种情况，重载为成员函数或非成员函数。

当重载为成员函数时，因为隐含一个指向当前成员的 `this` 指针作为参数，此时函数的参数个数与运算操作数相比少一个。

而当重载为非成员函数时，函数的参数个数与运算操作数相同。

其基本格式为（假设需要被重载的运算符为 `@`）：

```
1 class Example {
2     // 成员函数的例子
3     返回值 operator@(除本身外的参数) { /* ... */ }
4 };
5
6 // 非成员函数的例子
7 返回值 operator@(所有参与运算的参数) { /* ... */ }
```

下面将给出几个重载运算符的示例。

基本算数运算符

下面定义了一个二维向量结构体 `Vector2D` 并实现了相应的加法和内积的重载。

► 重载算数运算符的例子

自增自减运算符

自增自减运算符分为两类，前置 (`++a`) 和后置 (`a++`)。为了区分前后置运算符，重载后置运算时需要添加一个类型为 `int` 的空置形参。

可以将前置自增理解为调用 `operator++(a)` 或 `a.operator++()`，后置自增理解为调用 `operator++(a, 0)` 或 `a.operator++(0)`。

► 分别重载前后置自增运算符的例子

另外一点是，内置的自增自减运算符中，前置的运算符返回的是引用，而后置的运算符返回的是值。虽然重载后的运算符不必遵循这一限制，不过在语义上，仍然期望重载的运算符与内置的运算符在返回值的类型上保持一致。

对于类型 `T`，典型的重载自增运算符的定义如下：

重载定义（以 ++ 为例）	成员函数	非成员函数
前置	<code>T& T::operator++();</code>	<code>T& operator++(T& a);</code>
后置	<code>T T::operator++(int);</code>	<code>T operator++(T& a, int);</code>

函数调用运算符

函数调用运算符 `()` 只能重载为成员函数。通过对一个类重载 `()` 运算符，可以使该类的对象能像函数一样调用。

重载 `()` 运算符的一个常见应用是，将重载了 `()` 运算符的结构体作为自定义比较函数传入优先队列等 STL 容器中。

下面就是一个例子：给出 `n` 个学生的姓名和分数，按分数降序排序，分数相同者按姓名字典序升序排序，输出排名最靠前的人的姓名和分数。

下面定义了一个比较结构体，实现自定义优先队列的排序方式。

► 重载函数调用运算符的例子

比较运算符

在 `std::sort` 和一些 STL 容器中，需要用到 `<` 运算符。在使用自定义类型时，我们需要手动重载。

下面是一个例子，实现了和上一节相同的功能

► 重载比较运算符的例子

上面的代码将小于号重载为了成员函数，当然重载为非成员函数也是可以的。

► 重载为非成员函数

事实上，只要有了 `<` 运算符，则其他五个比较运算符的重载也可以很容易实现。

```
1 /* clang-format off */
2
3 // 下面的几种实现均将小于号重载为非成员函数
4
5 bool operator<(const T& lhs, const T& rhs) { /* 这里重载小于运算符 */ }
6 bool operator>(const T& lhs, const T& rhs) { return rhs < lhs; }
7 bool operator<=(const T& lhs, const T& rhs) { return !(lhs > rhs); }
8 bool operator>=(const T& lhs, const T& rhs) { return !(lhs < rhs); }
9 bool operator==(const T& lhs, const T& rhs) { return !(lhs < rhs) && !(lhs > rhs); }
10 bool operator!=(const T& lhs, const T& rhs) { return !(lhs == rhs); }
```

► 关于 C++20 下的三路比较运算符

参考资料与注释:

-
1. [运算符重载 - cppreference ↩↩](#)
 2. [用户定义字面量 - cppreference ↩](#)
 3. [比较运算符 #三路比较 - cppreference ↩](#)
 4. [默认比较 - cppreference ↩](#)
-

本页面最近更新: 2024/10/9 22:38:42, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [Enter-tainer](#), [lr1d](#), [ksyx](#), [qwqAutomaton](#), [shuzhouliu](#), [StudyingFather](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用